

WRRRA Cross Domain 1 0

Comparative Exact Validation in Games Power Grids and Scheduling

Minimal Sufficient State Residue and Constraint Preserving Rendering

Wonsik Choi

Independent Researcher

janefather@gmail.com

Version 1.0

27 September 2026

Abstract

WRRRA means Wonsik Reality Renderer Architecture. It was originally developed as a toy model for explaining how a world can execute through a small set of structural roles. This study tests a narrower computational claim. One fixed execution grammar was instantiated in three structurally different domains without changing the grammar: games, electric power grids, and machine scheduling. The common roles were Source, Law, State and Residue, Boundary, Common Carrier, Update, Renderer, Phenotype, and Ledger. Four tests were fixed before the two new finite experiments: state sufficiency, residue necessity, exact value preservation under a certified Renderer, and within-domain computation reduction.

The game baseline was reproduced from the published WRRRA Game 1.0 code. The rerun again enumerated 6,036,001 reachable states and 23,453,344 directed legal edges in 4 by 4 Connect K with k equal to 3. The certified game Renderer produced zero minimax value mismatches over all reachable states and reduced the root alpha beta search from 245,560 nodes to 32. The separate 3 by 3 simple ko Go census again found 132,161 reachable full states and 752 visible-state classes whose legal action sets depend on previous-board residue.

The new power-grid experiment exhaustively enumerated 4,409 states reachable over a six-step three-bus lossless DC trace with thermal-memory constraints. The certified Renderer reduced 28,517 labelled legal-action evaluations to 15,242, a 46.55 percent reduction, with zero exact cost-to-go mismatches. A same-visible-state witness had identical time and demand but different line-heat residue; its safe action sets differed and its exact remaining costs were 10 and 12. The new scheduling experiment exhaustively enumerated 1,103 states for six labelled jobs on two machines with sequence-dependent setup time. The Renderer reduced 4,188 job-machine action evaluations to 3,090, a 26.22 percent reduction, with zero exact makespan mismatches. A same-visible-state witness had identical remaining job types and machine availability but different last-family residue; its exact makespans were 10 and 11.

The central result is structural. The same execution grammar survived the same falsification tests in three finite domains, and every certified reduction preserved the exact objective in its declared scope. This establishes cross-domain portability of the tested architecture. It does not establish universal applicability, and the reduction magnitudes are not cross-domain performance scores because their work units differ.

Keywords: WRRRA, minimal sufficient state, residue, state abstraction, exact quotient, game search, DC power flow, scheduling, complex systems, explainable computation

Contents

- 1 Research Question and Main Result
- 2 The Fixed WRRRA Execution Grammar
- 3 Formal Tests and Exact Preservation
- 4 Validation Protocol
- 5 Game Domain
- 6 Power Grid Domain
- 7 Scheduling Domain
- 8 Cross Domain Comparison
- 9 Scientific Contribution
- 10 Scope and Limitations
- 11 Next Falsification Experiments
- 12 Reproducibility and Data Availability
- References
- Appendix A Residue Witnesses
- Appendix B Reproduced Game Invariants

1 Research Question and Main Result

The question is whether WRRR supplies a reusable execution architecture rather than a vocabulary that can be attached to any system after the fact. A vocabulary is easy to transfer because it makes no risky commitment. An architecture is harder to transfer because its components must support the same operations, failure tests, and accounting rules in each domain.

This report therefore fixes the WRRR grammar before comparing the new finite power-grid and scheduling experiments. Domain equations, state variables, actions, and objectives are allowed to change. The structural roles and pass conditions are not. A domain passes only if its complete state generates definite Law-permitted transitions, omission of a declared residue produces a reachable counterexample, a certified Renderer preserves the exact objective over the entire enumerated scope, and the reduction lowers the declared work count.

All three domains pass those tests. The result is stronger than three illustrative mappings. It contains exact state graphs, explicit same-visible-state counterexamples, unreduced and reduced solvers, zero-mismatch comparisons, machine-readable Ledgers, deterministic tests, and a rerun of the published game computation. The study also records a failure from earlier grid work: a more elaborate failure-memory controller did not repay its resource cost. WRRR is therefore not interpreted as a rule that memory or complexity should always be added. Residue is included only when the Law requires it, and additional memory remains a charged hypothesis.

The principal contribution has five parts.

- A fixed cross-domain contract maps the same nine structural roles into games, grids, and scheduling.
- Residue necessity is established constructively through reachable state pairs, not through metaphor.
- The new Renderer rule is certifiable because it removes only actions with the same future-relevant canonical successor and the same immediate objective contribution.
- Exactness and efficiency are separated. A reduction must first preserve every exact value in scope; only then is its work reduction reported.
- Small exact systems are connected to an existing 50,400-episode grid stress study without pretending that the stochastic study is an exhaustive proof.

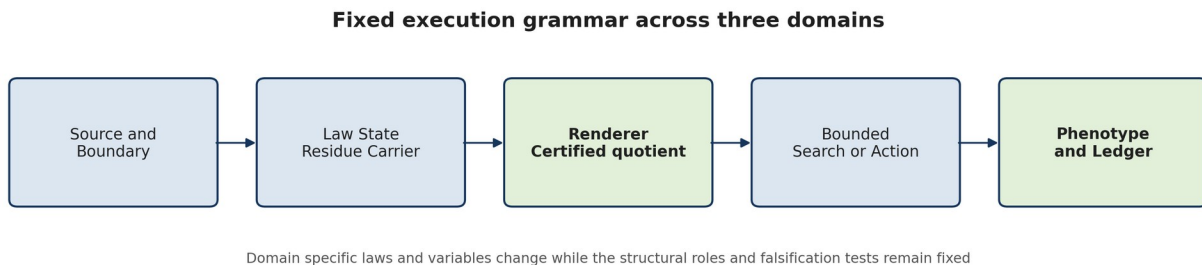


Figure 1 Fixed WRRR execution grammar used in all three domains

2 The Fixed WRRR Execution Grammar

WRRR is expanded here as Wonsik Reality Renderer Architecture. The architecture treats an executed system as a finite or bounded transformation of sources under laws and boundaries. The Renderer does not create a separate reality and does not predict a pre-existing future. It selects or compresses Law-permitted computations while preserving the declared objective when the reduction is certified.

The complete state at time t is separated into a visible component X and a residue R . Residue is current constraint information that cannot be recovered from the visible state alone but is required to determine an exact transition, legal action set, or value.

$$S_t = (X_t, R_t), \quad (S_{t+1}, c_t) = L_D(S_t, a_t), \quad a_t \in A_D(S_t)$$

In this notation D identifies the domain, A is the Law-permitted action set, L is the domain transition law, and c is the immediate objective contribution. A finite-horizon value is obtained by repeated application of this update and the stated terminal objective.

Table 1 maps the fixed roles to the three implementations. The mapping is deliberately operational. Every entry names a variable, transformation, or record used by code.

Table 1 Operational mapping of the fixed WRRR roles

WRRR role	Game	Power grid	Scheduling
Source	Board occupancy and side to move	Demand trace and available generation	Remaining labelled jobs
Law	Placement capture ko pass and terminal rules	Nodal balance DC flow generation bounds and thermal limit	Job assignment processing and setup rules
State	Board and turn	Time index and line heat	Remaining jobs and machine availability
Residue	Previous board and pass count for simple ko Go	Three line thermal-memory values	Last processed family on each machine
Boundary	Board size and stopping rule	Three buses three lines six time steps	Two machines six jobs
Common Carrier	Legal move and board transition	Net injection phase angle and line flow	Job-machine assignment and completion time
Update	Apply move capture and side change	Solve balance and flow then update heat	Add setup and processing time then remove job
Renderer	Certified game gates and ordering	Exact successor-and-cost quotient	Exact job-label and machine-symmetry quotient

WRRR role	Game	Power grid	Scheduling
Phenotype	Win loss draw and legal continuation	Surviving dispatch and cumulative cost	Completed schedule and makespan
Ledger	State census minimax and node counts	State edges costs residues and mismatch counts	State edges makespans residues and mismatch counts

The domain-specific content is substantial. Go ko, thermal line memory, and sequence-dependent setup are not the same phenomenon. Their common structural property is narrower: a visible description can omit present information that the transition law still needs. WRRR calls that information residue and charges it as part of state rather than hiding it in an implementation.

3 Formal Tests and Exact Preservation

State Sufficiency

A represented state is sufficient for the declared finite problem when it determines the Law-permitted action set, every successor state, each immediate objective contribution, and the terminal condition. This is an operational definition. It can be checked by a deterministic transition function and an exhaustive state graph.

Absolute minimality is a stronger claim. The new grid and scheduling experiments establish that the declared residue class is necessary by ablation, but they do not prove that every bit of every encoded field is globally irreducible. The report therefore distinguishes a sufficient full state from the narrower result that omission of the tested residue is false.

Residue Necessity

Let two reachable full states S_1 and S_2 have the same visible projection X . If their legal action sets, transition costs, or exact values differ, X alone is not sufficient. No exact controller or value function using only X can return the correct result for both states because it receives the same input for two different required outputs.

This test is constructive. Each domain must supply a reachable pair and the machine-readable histories or state fields that produce the conflict. Residue is therefore not a statement that history is philosophically important. It is the smallest presently retained trace needed to remove a specific ambiguity in the current transition law.

Certified Renderer

For the two new deterministic domains, the Renderer enumerates every Law-permitted action and computes its complete successor. It then forms equivalence classes using a canonical future-relevant successor and immediate cost.

$$a \sim_S a' \iff C_D(T_D(S, a)) = C_D(T_D(S, a'))$$

and $c(S, a) = c(S, a')$

The Renderer keeps one minimum-cost representative from each class. In the present experiments the costs are equal inside each retained class, so the representative choice is immaterial.

Exact preservation proposition. In a finite deterministic problem, if two actions have the same immediate objective contribution and their full successors have the same canonical representation under a value-preserving symmetry, removing all but one of them preserves the exact finite-horizon optimum.

Proof. At the terminal horizon, canonically equivalent states have the same terminal value by construction. Assume the claim holds for every state with at most k remaining transitions. For a state with k plus one transitions, equivalent actions contribute the same immediate cost and lead to successors with the same exact continuation value by the induction hypothesis. They therefore contribute the same candidate value to the Bellman minimum or maximum. Removing duplicates cannot change the optimum. Induction establishes the result for the finite horizon. The code does not rely only on this argument; it also compares the reduced and unreduced exact values at every reachable state.

The Game 1.0 Renderer uses domain-specific certified gates in Connect K rather than the generic successor quotient. Its published all-state census supplies the same empirical criterion: zero exact minimax-value mismatches over all 6,036,001 reachable states.

Computation Reduction

Each domain reports its own work unit. Game reports root alpha-beta nodes. The finite grid reports labelled legal-action evaluations across all reachable states. Scheduling reports labelled job-machine action evaluations across all reachable states. A reduction fraction is meaningful against the unreduced solver within one domain. It is not a common speed score across domains.

4 Validation Protocol

The analysis plan was frozen on 27 September 2026 before the new finite-grid and scheduling runs. It is prospective for those two experiments. The Game 1.0 and Grid Under Scarcity results existed beforehand and are handled as retrospective harmonization and reproduction. They are not relabelled as prospectively preregistered results.

Table 2 lists the four pass conditions.

Table 2 Common tests and failure conditions

Test	Pass condition	Falsification event
State sufficiency	Full state gives one exact transition and objective for every state-action pair	One represented state maps to conflicting legal transitions or exact values
Residue necessity	A reachable same-visible-state pair differs because the residue differs	No such pair exists in the declared exhaustive scope
Renderer exactness	Zero reduced-versus-full exact value mismatches at every reachable state	Any exact value mismatch caused by certified pruning
Computation reduction	Reduced work count is smaller in the declared unit	Equal or larger count after the reduction is exposed

The architectural claim also fails if a fourth domain can be implemented only by changing the common grammar rather than supplying domain-specific laws, variables, Renderer channels, and objectives. That condition makes future negative results informative.

The code writes every result to JSON and a compact CSV. Fractions in the finite DC model use exact rational arithmetic. Randomness is absent from the two new exact experiments. The earlier grid stress study used fixed seeds and common random numbers but remains a separate exploratory stochastic result.

5 Game Domain

Published Exact Baseline

WRRR Game 1.0 joined three stages of work. Game 0.1 constructed an Omok multiple-outlet position and a simple-ko residue witness. Game 0.2 enumerated finite Connect K and Go state spaces. WRRR Go 0.1 then reused the state-transition system in a playable MCTS engine with transparent Renderer channels and JSON Ledgers [2].

The exact Renderer test uses 4 by 4 Connect K with k equal to 3. The rules allow alternating placement, end at the first line of at least three stones, and include every reachable state from the empty board. The full graph contains 6,036,001 states and 23,453,344 directed legal edges. The exact solver and Renderer solver agree at all states. At the empty-board root, alpha-beta search evaluates 245,560 nodes without Renderer guidance, 15,686 with Renderer ordering, and 32 with the certified Renderer gate. The gate therefore removes 99.9870 percent of root nodes, a factor of 7,673.75, without changing the root value.

The residue test uses 3 by 3 Go with capture, suicide prohibition, simple ko, pass, and two-pass termination. The exhaustive graph contains 132,161 complete states and 420,710 directed legal edges. When previous-board residue is removed from the visible key, 20,538 visible classes contain multiple reachable residues. In 752 of those classes the legal action set differs. The shortest

retained witness reaches the same visible board and side to move by two legal histories, but a recapture at coordinate 0 comma 1 is legal after one history and blocked after the other because it recreates the previous board.

Independent Rerun in This Release

This release executed the published `wrra_game_0_2.py` source again. The new run took 246.96 seconds in the present environment. Seven predeclared invariants were compared with the published JSON rather than comparing environment-dependent elapsed times. Every invariant matched: Connect K reachable states, directed edges, all-state mismatches, baseline root nodes, Renderer root nodes, Go reachable states, and residue-sensitive classes.

The rerun matters because the game result supplies the largest exact state graph in the comparison. It also guards against a report that merely copies prior numbers. The source, published ledger, reproduced ledger, hashes, and field-by-field comparison are included in the package.

Playable Scale Transition

Game 1.0 also tested whether the verified transition system could support an actual agent when exhaustive search is no longer practical. Under a matched budget of 48 simulations per move on 5 by 5 Go, WRRR MCTS won 15 of 16 games against uniform-prior MCTS and 16 of 16 against random legal play. A 9 by 9 self-play game ended normally after 82 moves and two passes. These runs do not establish professional Go strength. Their role is architectural: the same Law, residue, Renderer, search, phenotype, and Ledger chain remains executable beyond the exact board sizes.

6 Power Grid Domain

Why the Grid Is an Independent Test

A power grid differs from a board game in three relevant ways. Its carrier is a continuous physical balance represented here by exact rational DC equations, actions may be physically distinct commands with the same aggregate effect, and a line can remain constrained by accumulated heat after the currently visible demand has changed. This gives the architecture a test in which residue is neither ko nor turn history.

The finite model has three buses and three unit-susceptance lines. Bus zero contains a slack generator with output from zero to four units. Bus two contains two labelled identical one-unit peakers. Bus one contains the demand and two labelled identical one-unit curtailment blocks. The six-step demand trace is 3, 4, 5, 4, 5, 3. The action is the on-off command for the two peakers and the curtail-or-serve command for the two demand blocks.

With bus zero as the reference, the reduced susceptance equation determines phase angles and line flows. Each line carries a thermal residue that decays by one half and accumulates only flow above its continuous rating.

$$B_{\text{red}}\theta = P, \quad f_{ij} = b_{ij}(\theta_i - \theta_j)$$

$$h'_{ij} = \frac{1}{2}h_{ij} + \max(0, |f_{ij}| - \bar{f}_{ij})$$

The line ratings are 2, 3 over 2, and 3 over 2. A transition is safe when every updated heat value is at most 3 over 2. The immediate operating cost is twelve units per curtailed block, three per peaker, and one per slack-generation unit. The exact objective is minimum total six-step cost subject to balance, generator bounds, and thermal survival.

The sufficient implemented state is the time index and three line-heat residues. The visible-state ablation retains only time and current demand. All arithmetic that affects equality, feasibility, and value uses Python Fraction objects, so the zero-mismatch result is not a floating-point tolerance claim.

Exact Result

The exhaustive graph contains 4,409 reachable states and 28,517 labelled safe action edges. The two labelled peakers and two labelled curtailment blocks create distinct commands that can have the same physical aggregate, complete successor, and cost. The certified Renderer retains one representative for each such equivalence class. Across every reachable state it evaluates 15,242 representatives, a reduction of 13,275 action evaluations or 46.55 percent. The full and reduced dynamic programs have the same exact cost at every state. The root cost is 30 in both programs.

The visible-state ablation fails. At time index four, current demand is five in two reachable states. One has line heat 0, 0, 0. The other has heat 13 over 12, 1 over 24, 0. The cool state permits fifteen labelled safe actions and has exact remaining cost 10. The warmer state blocks both single-peaker no-curtailment variants, permits thirteen labelled safe actions, and has exact remaining cost 12. A controller that sees only time and demand receives the same input in both states but must return different feasible sets and values.

This witness gives thermal memory a precise status. It is not a prediction of the next disturbance. It is a present line constraint produced by earlier loading and needed for the next Law update.

Relation to the Preserved Scarcity Study

The prior WRRA Grid Under Scarcity study tested a larger 14-bus toy system with 20 lines, five generators, three renewable sources, up to five storage locations, 96 time steps per episode, seven disturbance axes, and six controllers. It enforced the fixed equivalent resource budget

$$\text{generation MW} + 0.30 \text{ times storage MWh} + 8 \text{ times base controller compute} = 408.$$

Across 50,400 episodes, the memoryless local controller G0 had a survival-curve area of 85.34 percent and mean active compute of 114.04. The one-residual-state G2 controller had an area of 85.09 percent and mean active compute of 183.02. Because both had the same worst-axis pass level and G0 used fewer resources, the study selected G0 as the minimum sufficient architecture for that environment. Yet at compound stress 0.70, G2 survived 49.2 percent of episodes and G0 survived

11.7 percent, showing that a moderate reserve can help locally near a compound boundary even when it does not improve the formal pass threshold.

A separate 20,000-episode paired memory trial compared an otherwise similar no-memory reserve controller M0 with a failure-trace controller M1. At stress 0.70, M0 survived 83.6 percent and M1 survived 81.5 percent. The implemented memory did not repay its generation and computation cost. That negative result is retained because it sharpens the architecture: residue required by the Law and optional learned memory are different claims. The exact three-bus test proves that some thermal residue is necessary for the stated dynamics. It does not imply that every larger memory policy is useful.

7 Scheduling Domain

Model and State

The scheduling test uses six labelled jobs and two identical machines. Jobs A1, A2, and A3 belong to family A and require two time units. Jobs B1, B2, and B3 belong to family B and require three. A machine needs no setup before its first job or between jobs of the same family. Changing family adds two time units.

$$s(\ell, f) = 0 \text{ if } \ell = \emptyset \text{ or } \ell = f, \quad s(\ell, f) = 2 \text{ otherwise}$$

$$u'_m = u_m + s(\ell_m, f_j) + p_j$$

The full state contains the remaining labelled jobs and, for each machine, its availability time and last processed family. The objective is minimum final makespan. Every unscheduled job can be assigned to either machine, so the unreduced action count at a state is twice the number of remaining labelled jobs.

The Renderer uses two exact symmetries. Labels of jobs with the same family and duration do not change future processing. The two machines can be permuted when their complete availability and last-family states are permuted together. Each action is applied first, and its successor is canonicalized as a multiset of remaining job types plus a sorted pair of complete machine states. Actions with the same signature are exact duplicates for the continuation problem.

Exact Result

The full enumeration reaches 1,103 labelled states and contains 4,188 labelled job-machine action edges. The Renderer evaluates 3,090 representatives, removing 1,098 evaluations or 26.22 percent. The exact makespan agrees at every reachable state. Both solvers return a root makespan of 9.

Thirty-seven visible-state classes have residue-dependent next-transition profiles when last family is omitted. Six of those classes also contain different exact makespans. In the retained value witness, two states have the same two remaining A jobs and the same machine availability times 6 and 7. In one state both machines last processed B, so either A assignment requires a two-unit

setup and the exact makespan is 11. In the other state the machine available at time 7 last processed A, so assigning an A job there requires no setup and the exact makespan is 10.

This result matters because all actions remain syntactically legal. Residue is still necessary even when it changes cost rather than legality. A state definition can therefore fail by assigning one value to two current situations that require different exact continuation values.

8 Cross Domain Comparison

Table 3 reports the harmonized results. The game row combines the Connect K Renderer test with the separate Go residue census because Game 1.0 deliberately used the game best suited to each exact question. The new grid and scheduling rows apply both tests inside one finite model.

Table 3 Cross domain exact results

Domain	State graph	Residue-sensitive classes	Exactness	Work reduction
Game	6,036,001 states and 23,453,344 edges	752 in 3 by 3 Go	0 mismatches	245,560 to 32 nodes or 99.9870%
Power grid	4,409 states and 28,517 edges	1	0 mismatches	28,517 to 15,242 actions or 46.5512%
Scheduling	1,103 states and 4,188 edges	37	0 mismatches	4,188 to 3,090 actions or 26.2178%

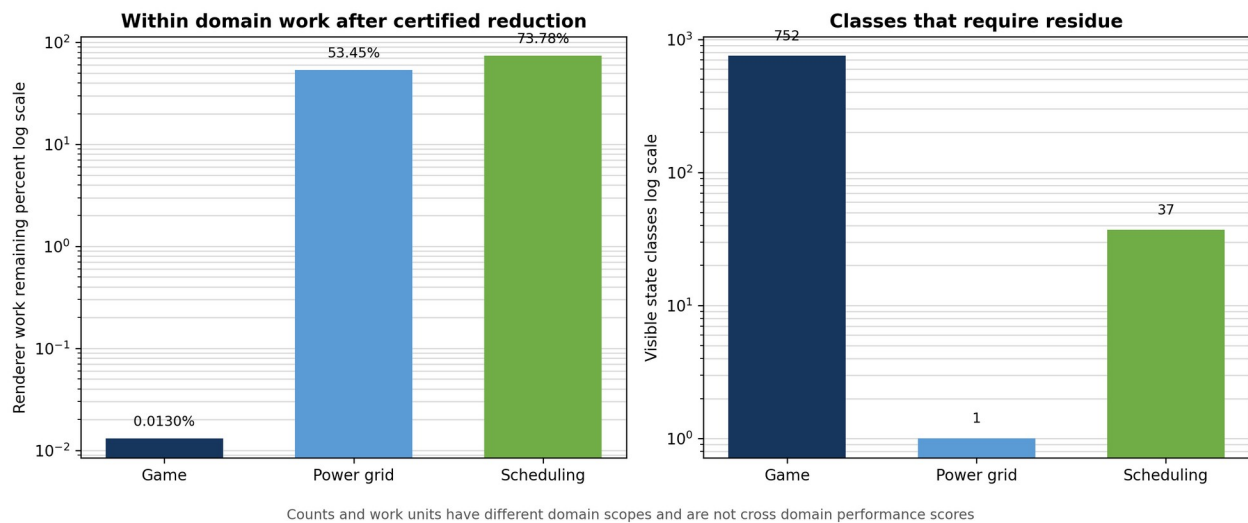


Figure 2 Within-domain Renderer work and residue-sensitive classes shown on logarithmic scales

Four conclusions follow directly from Table 3.

First, one structural grammar can carry exact executable content in all three domains. The mapping does not replace domain science. DC balance, Go ko, and setup time remain domain laws. WRRR supplies the common organization of those laws into state, residue, update, certified reduction, phenotype, and record.

Second, residue is not synonymous with long memory. The Go residue is one previous board, the grid residue is a three-component thermal state, and the scheduling residue is one family label per machine. Each is small relative to the full history. The evidence supports retaining the minimum present trace needed for an exact transition, not retaining the past without limit.

Third, certified reduction and heuristic search should remain separate. The present Renderer quotient has a proof obligation and an all-state mismatch test. Heuristic ranking may add practical value at larger scale, as the playable Go result suggests, but ranking success cannot be substituted for exact certification.

Fourth, the architecture can record negative results without contradiction. The grid memory trial showed that an optional failure trace cost more than it returned. That does not conflict with thermal residue necessity because the two memories serve different laws and are charged separately. This distinction prevents WRRR from becoming an unfalsifiable preference for additional structure.

9 Scientific Contribution

A Common Falsification Surface

The strongest contribution is the shared failure surface. Many cross-domain frameworks demonstrate that their terms can be mapped onto several examples. This work instead asks the same questions of each executable model. Does the state uniquely determine transitions? Can omission of residue be refuted by a reachable pair? Does the reduced solver match the full exact value at every state? Does the declared work count actually fall? A framework that fails one of these questions in a future domain has a recorded negative result rather than an interpretive escape.

Residue as a State Design Theorem

The three witnesses show three distinct forms of residue necessity.

- Go changes legal action membership.
- The grid changes both safe action membership and exact cost-to-go.
- Scheduling changes transition cost and exact makespan even though every assignment remains syntactically legal.

Together they show that state insufficiency can appear at legality, safety, transition cost, or value. This supplies a practical design rule for transparent agents: before adding predictive machinery, search for same-visible-state collisions in the transition ledger. If collisions exist, repair the present state definition first.

Exact Quotienting as a Renderer Baseline

The generic Renderer used in the new domains is intentionally conservative. It provides a lower-risk baseline against which more aggressive channels can be judged. A future heuristic Renderer

can be layered above the quotient, but any additional pruning must declare whether it is certified, empirically validated, or approximate. This taxonomy lets the architecture scale without erasing the difference between proof and performance.

The idea of grouping behaviorally equivalent states or transitions has precedents in model minimization and state abstraction [4]. The WRRRA contribution here is not the invention of equivalence quotienting alone. It is the placement of a certifiable quotient inside one cross-domain execution contract that also exposes residue, resource use, phenotype, and Ledger, followed by all-state comparisons in three domain instantiations.

Exact and External Scale Evidence in One Program

The project now contains two complementary evidence levels. Small systems permit exhaustive contradiction searches. Larger game and grid systems test whether the same execution chain still runs under bounded resources. Neither level replaces the other. Exhaustive toy systems establish exact local claims. Larger stochastic or search-based systems expose scaling behavior, resource tradeoffs, and failure modes that the small systems cannot represent.

10 Scope and Limitations

The result has a clear boundary.

The game Renderer reduction is exact for 4 by 4 Connect K with k equal to 3, not standard 15 by 15 Omok or 19 by 19 Go. The Go residue census is exact for the stated 3 by 3 simple-ko rules. The playable benchmarks measure execution under matched small simulation budgets and do not establish specialist-engine superiority.

The new grid is a deliberately finite DC model. It omits losses, voltage magnitude, reactive power, frequency dynamics, generator ramp history, protection logic, island restoration, uncertain forecasts, and AC feasibility. The thermal update is a transparent synthetic rule rather than a calibrated conductor model. The two peakers and two curtailment blocks are identical by construction, so part of the reduction comes from exposed command symmetry. The result proves exactness for that Law; it does not establish an operational controller for a real network.

The scheduling test uses two identical machines, two job families, deterministic processing times, and six jobs. It omits release dates, due dates, machine eligibility, stochastic arrivals, breakdowns, and nonidentical setup matrices. Job-label and machine symmetries are exact in this model but may disappear in a heterogeneous factory.

The term minimal is used operationally. The experiments charge every retained state field and refute selected omissions. They do not contain a complete information-theoretic proof that no alternative encoding with fewer bits exists. Future releases should reserve absolute minimality for cases with a formal lower bound or exhaustive representation comparison.

The reported reduction fractions are not commensurate. Game counts root alpha-beta nodes. Grid and scheduling sum labelled action evaluations over all reachable states. Wall-clock time is also

affected by language runtime, caching, and reporting overhead. The scientific comparison is zero mismatch under declared reduction and positive within-domain work reduction.

Finally, three successful domains do not prove that every complex system admits the same useful decomposition. The current result establishes a nontrivial reusable kernel and a method for trying to break it. Universality remains an open hypothesis.

11 Next Falsification Experiments

Power Grid

The next grid study should preserve the fixed-resource discipline while raising physical fidelity. A useful sequence is:

1. Replace the synthetic three-bus law with standard IEEE test cases and independently checked AC and DC power-flow implementations [5].
2. Add generator ramp residue, protection relay timers, and breaker lockout as separately ablated state fields.
3. Compare full-action, exact quotient, optimal power flow, and model-predictive controllers under the same generation, storage, communication, and computation budget.
4. Preregister $N - 1$ and compound contingencies, survival criteria, action costs, and the distinction between certified and heuristic pruning.
5. Test distribution shift with disturbance sequences not used to tune Renderer channels.

The strongest near-term target is not a larger survival percentage. It is a sharper necessity map: which physical residues change feasibility or value, which optional memories repay their resource cost, and which Renderer reductions remain exact after AC and protection constraints are introduced.

Scheduling

The next scheduling study should move from the six-job proof system to standard benchmark families. Sequence-dependent setup, unrelated machines, release dates, and online arrivals should be added one factor at a time. Exact small instances can continue to certify quotient rules; larger instances can compare node counts, optimality gaps, and ledger explanations with branch and bound or mixed-integer baselines. Established scheduling notation and benchmark practice provide an external comparison surface [6].

Games

The game branch should keep the present separation between rule correctness, exact small-board proof, and playable search. Larger exact Connect K variants can test where the certified gates stop producing large reductions. Go experiments should test positional and situational superko, because each rule changes the residue required for exact legality. MCTS comparisons should report matched simulations, paired colours, uncertainty intervals, and complete game records [7].

Cross Domain Theorem and Fourth Domain

The next architectural paper should formalize a typed interface for Law, State, Residue, canonical successor, objective, and Ledger. It should then attempt a fourth domain chosen before implementation, such as packet routing or inventory control. A failed instantiation would be valuable if it identifies which structural role is missing or which certification condition cannot be met.

12 Reproducibility and Data Availability

The release contains the prospective analysis plan, source code, standard-library tests, published baseline Ledgers, the reproduced game Ledger, new exact results, summary CSV, figures, report, licenses, and a SHA-256 manifest. The main exact run is deterministic.

From the release root, the new comparison is reproduced with:

```
python src/wrra_cross_domain_1_0.py --game-json
baselines/wrra_game_0_2_results_published.json --game-reproduced-json
results/reproduced_game_0_2_results.json --grid-scarcity-json
baselines/wrra_grid_scarcity_stress_summary_published.json --output-dir
results/reproduced_cross_domain
```

The deterministic tests are run with:

```
python -m unittest discover -s tests -v
```

The complete Game 0.2 rerun is produced with:

```
python vendor/wrra_game_0_2.py --output results/reproduced_game_0_2_results.json
```

Code in src, tests, tools, and vendor is released under the MIT License. Reports, figures, protocol, documentation, and machine-readable results are released under Creative Commons Attribution 4.0 International.

References

- [1] Choi W. WRRR Core 1.0. Zenodo. 2026. DOI 10.5281/zenodo.22650956.
- [2] Choi W. WRRR Game 1.0 Minimal Sufficient State Exhaustive Validation and a Playable Go Engine. Zenodo. 2026. DOI 10.5281/zenodo.22985378.
- [3] Choi W. WRRR Grid research series. Zenodo records 22330256, 22331940, and 22447864. 2026.
- [4] Givan R, Dean T, Greig M. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence*. 2003;147:163-223.
- [5] Zimmerman RD, Murillo-Sanchez CE, Thomas RJ. MATPOWER Steady-State Operations Planning and Analysis Tools for Power Systems Research and Education. *IEEE Transactions on Power Systems*. 2011;26(1):12-19. DOI 10.1109/TPWRS.2010.2051168.

[6] Graham RL, Lawler EL, Lenstra JK, Rinnooy Kan AHG. Optimization and Approximation in Deterministic Sequencing and Scheduling A Survey. Annals of Discrete Mathematics. 1979;5:287-326. DOI 10.1016/S0167-5060(08)70356-X.

[7] Kocsis L, Szepesvari C. Bandit Based Monte Carlo Planning. Machine Learning ECML 2006. Lecture Notes in Computer Science 4212:282-293. DOI 10.1007/11871842_29.

Appendix A Residue Witnesses

Table A1 Same-visible-state witnesses

Domain	Shared visible state	Residue A	Residue B	Exact consequence
Go	Board B dot B slash W B dot slash dots with White to move and pass count zero	Previous board permits 0 comma 1	Previous board makes 0 comma 1 recreate the prior board	Legal action sets differ
Power grid	Time four and demand five	Heat 0 0 0	Heat 13 over 12 1 over 24 0	Fifteen versus thirteen labelled safe actions and remaining cost 10 versus 12
Scheduling	Two A jobs remain and machine availability is 6 and 7	Last families B and B	Last families A and B	Exact makespan 11 versus 10

The witnesses cover three failure modes of a visible-state-only model: an illegal move can be admitted, an unsafe physical action can be admitted, or a legal action can receive the wrong exact value.

Appendix B Reproduced Game Invariants

Table B1 Published and reproduced Game 0.2 values

Invariant	Published	Reproduced	Match
Connect K reachable states	6,036,001	6,036,001	Yes
Connect K directed legal edges	23,453,344	23,453,344	Yes
Connect K all-state value mismatches	0	0	Yes
Connect K root baseline nodes	245,560	245,560	Yes
Connect K root Renderer nodes	32	32	Yes

Invariant	Published	Reproduced	Match
Go reachable full states	132,161	132,161	Yes
Go residue-sensitive visible classes	752	752	Yes

Elapsed time is excluded from identity checking because it depends on the execution environment. The published and reproduced JSON files have different hashes for that reason, while all declared scientific invariants match.